

吳承儒

後端工程師

新竹市 | 聯絡方式請透過 github.com/hotrabbit1016

關於我

5 年後端開發，都在線上娛樂平台。主力是 C#/.NET，寫過玩家下注、結算、營運後台這類典型的 API 與即時推播服務；另外也用 Go 做過稽核資料管道，用 Python 做實體賽道的硬體控制與 AI 影像判定。

比較不一樣的地方是我的工作同時橫跨軟硬兩端——一端是雲上的分散式服務，另一端是會真的壞掉的實體設備與不穩定的 AI 推論。這讓我很習慣處理「外部訊號不可信」的問題，也讓我在寫服務時比較在意失效路徑而不只是快樂路徑。

技術能力

後端	C# / .NET 10、ASP.NET Core、EF Core、Go (Gin、Ent、Uber Fx)、Python
即時與訊息	SignalR (Hub 設計、重連治理)、WebSocket、RabbitMQ、NATS
資料儲存	MySQL、Redis (Lua script、Stream、分散式鎖)、ClickHouse、Elasticsearch
部署維運	Docker、Kubernetes、Helm、GitLab CI、ArgoCD、Istio、OpenTelemetry
AI 與影像	YOLO、Roboflow、PyTorch、TensorFlow、OpenCV、ONNX
硬體介接	SPI (MCP23S17)、UART / RF、Raspberry Pi、基礎電路焊接與電路板設計討論

工作經歷

泰鑫軟體 後端工程師

2024/01 – 現在

公司經營者更換，原團隊與整條產品線一起移轉過來 (2024/07 完成正式聘用)。負責桌台客戶端重構、營運後台、聊天平台稽核功能，以及實體賽道控制與 AI 賽果判定。

凱駁國際、鼎順數位科技 後端工程師

2021/04 – 2023/12

前後隸屬兩間公司，實際經營者與團隊相同，工作內容連續未中斷。從桌台系統的報表轉檔與狀態處理開始，後期負責玩家端下注服務與桌台現場管理服務。

這五年都是同一個團隊、同一條產品線。前期任職的兩間公司背後是同一位經營者，2024 年才真正換了經營者，團隊與系統一起移轉過去，所以下列專案期間會跨越任職紀錄。

專案經歷

線上娛樂平台後端服務群

2022 – 2026

.NET SignalR RabbitMQ Redis EF Core MySQL

三個服務串起玩家端到營運端的鏈路，我負責其中的交易流程、即時推播與後台功能。

- **PlayerGameServer**——玩家下注與取消下注的交易流程，結算後以 SignalR 即時推播通知；跨服務的狀態變更走 MQ 事件處理。
- **桌台營運後台**——營運與客服端的管理後台，包含權限稽核、局結果更正流程，以及第三方開獎結果的驗證機制；管理員登入以 Otp.NET 加上二階段驗證。
- **桌台現場管理服務**——荷官與攝影棚端的 SignalR Hub，負責賽局流程的指令下發。開牌結果要經過 AI 辨識與現場人工雙重確認才算成立。

桌台客戶端重構

2025 – 2026

.NET 10 Redis Kubernetes xUnit

把原本的 WPF 桌面程式改成可容器化的 Console 服務，並解掉多 Pod 部署時同一張桌被重複接管的問題。主程式與測試由我獨立設計與實作，共用函式庫則是團隊共同維護。

- 以 Redis + Lua script 實作桌台租約。用 Lua 而不是 GET-then-SET，是為了消掉 check-then-act 之間的 race——同一張桌被兩個 Pod 同時接管會造成重複下注，那是金流事故。
- Supervisor-Worker 調度：每桌一個 worker 與獨立 DI scope，單桌異常不影響其他桌。續租失敗就主動停掉 worker，寧可短暫沒人服務，也不能兩個 Pod 同時操作同一張桌。
- 桌台生命週期改用 State Pattern，狀態分派收斂成單一 factory，未處理到的狀態一律落進錯誤復原。
- 補上覆蓋 8 個桌台狀態與調度核心的單元測試，並把時間相關邏輯改走 TimeProvider，倒數與掃描迴圈才能用假時鐘快速測試。
- 清掉版控裡的明文密碼與連線字串，改由 K8s Secret 注入。

Go ClickHouse Redis Stream Gin Uber Fx

多租戶聊天平台的稽核紀錄功能，從方案選型、設計文件到上線由我獨立完成。

- 選型時放棄「擴充既有 audit_logs」這個比較快的做法，改建獨立的 operation_logs 表。工時大約多一倍，換到的是業務語意跟 HTTP raw log 徹底分開，而且完全不動既有管道。
- 寫入走 Redis Stream 加 Consumer Group 再落 ClickHouse，稽核不拖慢主請求；ClickHouse 用 MergeTree 按日分區、TTL 自動過期。
- Middleware 負責 HTTP 層語意（角色、選單、操作類型），Service 層負責變更前後的狀態快照，兩條路徑匯流到同一個 Stream 再合併寫入。
- 把 OperationType 與 Menu 做成實作 encoding.TextUnmarshaler 的封閉型別。一份實作同時擋住 Gin form binding 與 Redis Stream 的髒資料，非法值直接 400，而不是靜默回一個空結果。
- 查詢區間的界線踩到時區問題：QA 回報同一天查得到、但凌晨那幾個小時被擋掉。根因是前端以當地日界回推天數、後端跑 UTC，兩邊差了將近一天。最後決定後端不引入業務時區，只做寬鬆的 backstop 並多留一天緩衝吸收時差，精準界線交前端把關。
- 快照一開始記 base enum 跟裸 ID，後來全改成角色名與資源名。稽核紀錄的讀者是稽核人員，記 role=3 對他們沒有意義。

實體彈珠賽道控制與韌體

2024 – 2026

Python SPI UART/RF Raspberry Pi Flask OpenTelemetry

控制 8 條實體賽道（含一條實體測試賽道）的自動化系統，從硬體驅動到 Web 監控介面由我獨立開發。除了控制軟體，也寫韌體、跟外部廠商一起討論電路板設計、做基礎焊接。

- 硬體抽象層以 SPI 驅動 MCP23S17 I/O Expander，讀取最多 28 路光學感測器，並自動偵測新舊兩代感測板，讓同一份程式碼吃兩種硬體。
- RF 閘門驅動是最難的一塊，實體環境什麼都會壞，最後疊了六層防禦：指數退避重試、port 重新初始化的斷路器（強制 2 秒冷卻，否則 USB 連續斷線會引發 reinit 風暴級聯崩潰）、明確拉低 DTR/RTS 避免模組被控制線 toggle 重置、每秒探測 CTS 防止 USB autosuspend 讓裝置睡著、atexit 收尾避免重啟後留下 zombie handle，以及 write timeout 防止 TX buffer 卡住時無限阻塞。
- 賽道流程用 Strategy 加 Factory，8 條賽道各有自己的開賽條件與結束判定，新增一條只要加一個 flow 並註冊，不動主控迴圈。
- 偵測不到硬體時自動退回模擬模式，開發者在沒有 SPI 的 Mac 上也能跑完整流程。
- 依賽事事件切換 OBS 場景，把遊戲邏輯與直播導播串起來。三個進程分別以 systemd 常駐，監控介面掛掉不會影響賽道控制。

AI 影像辨識賽果判定

2022 – 2026

Python YOLO Roboflow PyTorch TensorFlow OpenCV ONNX FastAPI PyQt5

分兩個階段：前期做真人棋牌類的視覺辨識，取代原本的條碼掃描；後期做彈珠賽事的即時排名判定。兩者都已上線運行。

- 彈珠賽果以 Roboflow 標註並訓練 YOLO 模型，把每顆球訓練成獨立 class，排名系統就能直接用 class 辨識球體，繞過多物件追蹤的 ID switch 問題。
- 落球判定做成四狀態機（ACTIVE → PENDING → CONFIRMING → DROPPED）。關鍵決定是所有閾值都用秒、不用幀數——YOLO 推論的 fps 會隨負載浮動，拿幀數當基準遲早會漂掉。
- 二段時間確認用來容忍偶發的漏偵測，存活時間門檻濾掉開局瞬間的雜訊誤判，另有 stale 機制清除幽靈軌跡。
- 排名依據取「最後一次被偵測到的時間」而不是確認落球的時刻，確認延遲才不會影響排名的公平性。
- 對外以 FastAPI 提供賽果查詢，現場操作介面用 PyQt5；推論放在獨立執行緒，避免卡住 UI。

DS88 Webhook 對接模擬器

2026

Python Docker Helm GitLab CI ArgoCD

從零做到 K8s 常駐的內部工具，讓後端與 QA 在拿不到廠商測試環境時也能驗證 webhook 對接。刻意只用標準庫寫，QA 不必裝任何套件；自帶精簡 Helm chart 與三階段 CI pipeline。部署前把憑證從 K8s Secret 改成由使用者在網頁自填、存各自瀏覽器，並在 README 寫明這個工具沒有身份驗證機制、只能放測試環境憑證。

學歷

國立中興大學 電機工程學系

語言

中文（母語）、英文（TOEIC 770）